

Sundog Exam2 分析 + Exam1/Exam2 統合復習

CCDV-F

Exam2: 83%合格(44/53)・誤答8・Silent miss4・Exam1(81%)と統合・作成 2026-07-20・姉妹: [Exam1弱点分析](#)

Exam2 サマリー 弱点は「振り子」した

① 最大弱点 (Exam1→急落)

D1 Agents 50%

Exam1 88%→50%に半減。
Agent SDK・workflow
patterns・Workflow tool の選別
で4問失点。配点14.7%でRO急
所。

② 得意→崩落

D6 Prompt 67%

Exam1 100%→67%。context
editing・Structured Outputs の
境界知識が浅かった。
prefill≠caching≠context
editing。

③ 克服 (Exam1弱点)

D8/D5/D7 100%

Exam1 の
D8(50%)/D5(78%)/D7(75%) は
全て100%に克服。ツール原語・
新モデル機構は定着。→ 弱点が
別領域に移動。

核心: 弱点の振り子現象。 Exam1 の弱点(D8/D5/D7)を意識して臨んだ結果、今度は **D1/D6/D3 (Exam1で得意) が崩落**。部分的復習では弱点が移動するだけ。**106問全体を統合し、トピック単位で根本知識を固める** 必要がある — これが本HTMLの目的。

Part A — Exam2 領域別成績 + Exam1 比較

Domain	配点	題数	Exam2	Exam1	変化	Exam2 誤答	診断
D1 Agents & Workflows	14.7%	8	50%	88%	↓ -38	6,12,13,44	P0 知識が浅い
D2 Applications & Integration	33.1%	17	88%	82%	↑ +6	47 (+1skip)	P1 安定・迷い多い
D3 Claude Code	3.1%	2	50%	100%	↓ -50	20	P2 題数少
D4 Eval, Testing	2.6%	1	100%	100%	→	—	維持
D5 Model Selection	16.8%	9	100%	78%	↑ 克服	—	P1 確信不足
D6 Prompt & Context	11.0%	6	67%	100%	↓ -33	18,51	P1 境界知識
D7 Security & Safety	8.1%	4	100%	75%	↑ 克服	—	維持
D8 Tools & MCPs	10.6%	6	100%	50%	↑ 克服	—	P1 要メンテ

Exam2の分布: 誤答8 = D1(4) + D6(2) + D3(1) + D2(1)。D1 Agents に**半数が集中**。Exam1 の誤答分布(D8:3, D2:3, D5:2, D7:1, D1:1)とは**ほぼ逆転**。+ skipped 1 (D2、時間切れ)。

Part B — Exam2 誤答8問 詳細解説 (ギャップ6分類)

「あなた」=選んだ誤答、「正解」=正解選択肢、「漏選」=正解だが未選択。**Silent miss**=自信があったが誤、**Caught**=★marked (自覚済) で誤。

分類 ① Agent SDK / agentic loop の選別 (Q6, Q12)

Q6

D1 Agents

単選

Silent miss

ゲームスタジオのビルドチーム。内部デスクトップutility。ローカルcheckout を検査・編集・標準coding tools・UI に進捗 stream・tool call 前に決定的検証。**tool 実行 loop は自前実装したくない。**

~~あなた Claude Managed Agents を cloud sandbox にデプロイ・各checkout をコピー・session events を stream~~

正解 Claude Agent SDK を組み込み・file tools 有効化・SDK messages を stream・hooks 追加

ギャップ: 3つの wiring path (M2 m2:734) の選別。Managed Agents = **クラウド sandbox** (本件はローカル直操作に不適)。Client SDK = tool loop **自前実装要**。**Agent SDK** = プロセス内で Claude Code の loop・built-in tools・hooks を組み込み。→「ローカル + loop 自前不要」なら Agent SDK 一択。

📖 M2 m2:734 (3 wiring paths)・m2:1401 (Agent SDK 定義)

Q12

D1 Agents

単選

Caught

海事保険。TypeScript desktop app。ローカルcheckout・built-in coding tools・autonomous loop・UI stream。loop 構築回避 + 各 laptop への CC 別途インストール回避。

~~あなた Anthropic Client SDK インストール・Messages API で tools を自動実行~~

正解 @anthropic-ai/claude-agent-sdk インストール・streaming agent loop 使用

ギャップ: Client SDK は Messages API の**薄い wrapper** (agent loop / built-in coding tools なし・自前 harness 必要)。Agent SDK の TS 版 `@anthropic-ai/claude-agent-sdk` は **native Claude Code binary を同梱** (CC 別途不要)。旧 "Claude Code SDK" は legacy 名。

📖 M2 m2:1401 (SDK 定義) / M3 m3:319/339 (Managed Agents beta)

分類 ② Workflow / orchestration pattern の選別 (Q13, Q44)

Q13

D1 Agents

単選

Caught

公選弁護人の e-discovery。各 case は**同じ intake memo** で終わる。中間作業は検査後**異なり得る** (timeline / 証人比較 / 不一致check / 索引)。worker 役割は事前既知。**open-ended より予測可能性重視。**

~~あなた simple routing workflow・最初の文書のみで1専門家に送る~~

正解 orchestrator-workers workflow・監督 LLM が関連 worker 分析を割当・出力統合

ギャップ: workflow patterns ("Building Effective Agents" m2:1386) の選別。**routing**=1入力を1パスに分類 (本件は1 case に複数分析が必要で不適)。**orchestrator-workers**=動的に subtask 分解しつつ予測可能な最終構造を保持 = 本件の sweet spot。完全 autonomous は over-kill。

📖 M2 m2:1386 / M4 m4:617/631 (orchestrator-worker・コスト)

Q44

D1 Agents

単選

Silent miss

臨床試験 data platform。各 migration で60 の独立 check ・ findings 比較 ・ 統合 plan。main 会話は coordination に集中 ・ 全検索結果/中間 transcript 蓄積回避。

~~あなた~~ main agent が各 check を1つずつ subagent 呼出

正解 Workflow tool で orchestration script から独立 check を実行

ギャップ: subagents(少数 ・ 隔離 context) vs Workflow tool(大規模 fan-out ・ orchestration を script 化 ・ 会話外実行) の使い分け (m2:1409 / m3:211)。数十の独立 check は Workflow tool。1個ずつ委任は会話 loop に orchestration が残り latency/coordination overhead。

📄 M2 m2:1409 (Subagent) / M3 m3:211 (Subagents) / 公式: Dynamic workflows

分類 ③ Context 管理メカニズム (Q18)

Q18

D6 Prompt & Context

単選

Caught

医薬品安全監視。Claude triage agent が冗長 scanner logs を返す tool を呼ぶ。findings 抽出後 ・ 古い tool 出力は不要。ただし完全 audit 履歴は自前 DB 保持。client 側履歴を書き換えずに API で Claude の見える情報を減らしたい。

~~あなた~~ audit DB から古い tool result messages を削除

正解 context editing ・ request の tool result clearing を有効化 (`clear_tool_uses_20250919`)

ギャップ: 3つの区別。① context editing=server-side で Claude に届く前に適用 (client は完全履歴保持可)。② prompt caching=cost/latency 最適化 (cached token は文脈を占有 ・ 刈りでない)。③ application storage=監査DB (削除は監査要件と衝突)。→ 「DB保持 + Claudeの見える文脈だけ削る」 = context editing 一択。

📄 公式 doc: Context windows (context editing / clear_tool_uses_20250919) — **Prep未カバー**

分類 ④ Claude Code 実行モード (Q20)

Q20

D3 Claude Code

複選(2)

Silent miss

景観建築 firm。scheduled local script・Claude Code で再生 OpenAPI files を検査。interactive terminal を開かない・別プロセスが parse 可能な出力・team 定義の agent turn 数で停止。Which TWO。

正解✓ `claude -p` / `--print` で検査 prompt を実行 (あなたは正選)

あなたは interactive で `claude` 起動 + `--max-turns` 設定後 prompt 入力

正解✓ `--output-format json` / `stream-json` + `--max-turns` 設定

ギャップ: `print mode`(`-p` / `--print`) が非対話・自動化専用。`--max-turns` と `--output-format` は `print mode` 専用 (interactive では適用外・要件の「非対話」にも違反)。`print mode` はデフォルトで turn 上限なしなので cap が必要。

📄 公式 doc: Run Claude Code programmatically・CLI reference — Prep未カバー

分類 ④ 並列隔離パターン (Q47)

Q47

D2 Apps & Integration

単選

Silent miss

都市計画 software vendor。Claude Code で3競合 refactor を試行。各 attempt は多数 file 修正・test 実行。diff を side by side 比較・human が最良設計選択前に attempt が別を上書き回避。

あなたは ~~1 file 編集を全計画完了まで無効化・1統合 passで全計画変更を適用~~

正解各 attempt ごとに別 git worktree・独立 Claude Code session を各 checkout で実行

ギャップ: `git worktrees`=filesystem + branch で隔離 (session は directory 紐付・各実験が自由に編集/test・diff 比較容易)。branch switch は会話履歴が残留し混入。same checkout は衝突。統合 pass は「比較」の目的を損う。→ 並列実験は worktree 一択。

📄 公式 doc: Claude Code overview・Create custom subagents — Prep未カバー

分類 ◎ Structured Outputs の境界 (Q51)

Q51 D6 Prompt & Context 単選 Caught

保険 underwriting。Claude Opus 4.8 で claim notes 分類・下游 routing。legacy queue 名 `Review Needed` / `Review needed` (大文字小文字のみ相違・別 team)。parse 可能 JSON・出力取扱前提の誤 routing 回避。

~~あなた~~ prompt で JSON 要求・最初の object parse・syntax error のみ retry

正解 `output_config.format` で stable な non-casing status codes 露出・stop reasons check・codes を queues に map

ギャップ: ① Structured Outputs (`output_config.format` +JSON Schema) が schema 保証の正道 (prompt+parse+retry は非公式)。② ただし **enum の casing は保証されない** (大文字小文字のみ相違は誤 route) → stable code に置換。③ **stop_reason** (refusal/max_tokens) check 必須。④ **prefill は JSON と両立不可**・Opus 4.8 は prefill 非対応。

📘 M2 m2:121/136/141 (output constraints / output_config.format)

Part B 補足 — Silent miss 4 問 (自信があったが間違い)

誤答8のうち ★marked (自覚済) は4問 (Q12,13,18,51)。残り 4問は自信があったのに間違えた「Silent miss」。自覚精度は Exam1 (silent 6 / caught 4) から改善したが、本番で最も危険なのはこの Silent。

問	Domain	見落とした知識	潰す場所
Q6	D1	Agent SDK (ローカル・loop組込) vs Managed Agents (クラウド sandbox)	M2 m2:734
Q20	D3	print mode 専用 flag (<code>--max-turns</code> / <code>--output-format</code>)	公式 doc
Q44	D1	Workflow tool (大規模 fan-out) vs subagents (少数)	M2 m2:1409
Q47	D2	git worktrees (filesystem+branch 隔離)	公式 doc

Silent miss の共通: 「自明に見える問題」で最もらしい選択肢 (Managed Agents / interactive+max-turns / subagent 1個ずつ / 統合 pass) に引っかかった。いずれも **部分正しいが要件を満たさない**。→ 自信があるほど選択肢を **全つぶし** する習慣が対策。

Part C — Exam1/Exam2 統合分析（106 問・18 誤答）

2 試験を統合（Exam1: 43/53・Exam2: 44/53 = 計 87/106 = 82%）。誤答 18 をトピックグループで整理し、根本知識の弱点を浮かび上がらせる。

重み付き優先度マトリクス（配点 × 不正率）

	高配点（≥11%）	低配点（<11%）
低正答率（<80%）	● 最優先 D1 Agents 14.7% / 69% D8 Tools&MCPs 10.6% / 75%	● 知識ギャップ D3 Claude Code 3.1% / 75%
高正答率（≥80%）	● 確信不足・維持 D2 Apps 33.1% / 85% D5 Model Sel 16.8% / 89% D6 Prompt 11.0% / 83%	● 維持 D7 Security 8.1% / 88% D4 Eval 2.6% / 100%

Domain	配点	2試験題	2試験誤	正答率	配点×不正	優先
D1 Agents & Workflows	14.7%	16	5	69%	4.6	P0
D2 Applications & Integration	33.1%	34	5	85%	5.0	P1
D8 Tools & MCPs	10.6%	12	3	75%	2.7	P1
D6 Prompt & Context	11.0%	12	2	83%	1.9	P1
D5 Model Selection	16.8%	18	2	89%	1.8	P1
D7 Security	8.1%	8	1	88%	1.0	P2
D3 Claude Code	3.1%	4	1	75%	0.8	P2
D4 Eval	2.6%	2	0	100%	0	維持

誤答 18 問のトピックグループ化

トピックグループ	問数	該当問	主戦場
① ツール/Agent 原語の選別 client tool / MCP / Skill / server tool / Agent SDK / Client SDK / Managed Agents	5	Exam1: 22,23,41 / Exam2: 6,12	M2 ・ M3
② Workflow / orchestration pattern orchestrator-workers / routing / chaining / Workflow tool / subagents / query vs ClaudeSDKClient	3	Exam1: 20 / Exam2: 13,44	M2 ・ M4
③ Prompt/Context 機構 prefill / context editing / Structured Outputs	3	Exam1: 33 / Exam2: 18,51	M2 ・ 公 式
④ Claude Code 実行/設定 CLI override / print mode / worktrees / 設定階層	3	Exam1: 44 / Exam2: 20,47	公式 doc
⑤ 新モデル機構 (Exam2 で克服) effort / Haiku routing / thinking 課金 / cache_control	2	Exam1: 15,34	—
⑥ Skill versioning	1	Exam1: 5	M3 ・ M5
⑦ Security (injection/trust) (Exam2 で克服)	1	Exam1: 50	—

統合の核心: 18 誤答の過半数 (11問) が グループ①+②+③ = 「Agent/Tool/Prompt の選別・使い分け」に集約される。これは全て **Prep M2** (+ 一部 M3 ・ 公式 doc) で潰せる。一方グループ④ (Claude Code 実行) 3 問は**Prep 未カバー**で公式 doc 必須。グループ⑤⑦ (4問) は Exam2 で既に克服済み。

marked vs incorrect のギャップ診断 (Exam2)

Domain	marked (自覚)	incorrect (実害)	診断
D2 Apps	7	1 (silent)	知識あるが迷い多い → 演習で確度
D5 Model Selection	6	0	6問迷って全問正解 = 確信不足 (知識はある)
D6 Prompt	4	2	半分落とす → 境界知識を補強

Domain	marked（自覚）	incorrect（実害）	診断
D1 Agents	4	4	迷って落とす + 無自覚に落とす = 知識が浅い
D8 Tools/MCPs	3	0	Exam1 弱点→Exam2 安定
D7 Security	2	0	安定

診断の意味: D5 と D2 は「迷ったが正解」が多く**知識はあるが確信不足**（演習量で解決）。D1 は「迷って落とす + Silent で落とす」が混在し**知識そのものが浅い**（Prep M2 の精読で解決）。→ D1 は「読む」、D5/D2 は「解く」。

Part D — 腰を据えた復習計画（核心）

Exam1 弱点は Exam2 で克服したが、別領域が崩れた。本番前は領域単位でなくトピック単位で根本知識を固める。Prep M2 が主戦場、3 トピックは公式 doc 補完。

● P0 — 本番前に必ず完了 目安: 約120分

① Prep M2 徹底 — Agent / Workflow / Tool の選別 約60分

- 対象: トピックグループ①②③（11問）+ D1 知識不足の根本解決
- 読む箇所（行番号確定）：
 - m2:734 — 3 つの wiring paths（Messages API 直接 / Agent SDK / Managed Agents） = Q6/Q12 の核心
 - m2:1386 — "Building Effective Agents"（chaining/routing/parallelization/evaluator-optimizer） = Q13 の核心
 - m2:1401 — Claude Agent SDK 定義（@anthropic-ai/claude-agent-sdk ・ Anthropic SDK との区別）
 - m2:1409 — Subagent 定義（隔離 context ・ 少数委任） = Q44 の核心（vs Workflow tool）
 - m2:121/136/141 — output constraints / output_config.format（Structured Outputs） = Q51 の核心
- M3 補強: m3:211（Subagents）・ m3:319/339（Managed Agents beta）
- M4 補強: m4:617/631（orchestrator-worker pattern ・ コスト15倍例） = Q13 の裏付け

② 公式 doc 補完 — Prep 未カバーの 3 トピック 約45分

- context editing / clear_tool_uses_20250919（Q18）→ 公式 doc 「Context windows」。server-side 編集 vs caching vs application storage の 3 区別
- Claude Code print mode（Q20）→ 公式 doc 「Run Claude Code programmatically」 + 「CLI reference」。-p / --max-turns / --output-format json/stream-json
- git worktrees（Q47）→ 公式 doc 「Claude overview」「Create custom subagents」。filesystem+branch 隔離 ・ 会話履歴との分離

注意: この 3 トピックは Prep コースに記述がない（grep 検証済み）。放置すると本番でも Silent miss になる。公式 doc を 1枚ずつ 読んで自分の言葉で要約すること。

● P1 — 演習で確度アップ 目安: 約40分

◎ D5 Model Selection — 「迷って正解」を「確信して正解」へ

- Exam2 で100%だが**6問 marked**=確信不足。知識はあるので**演習量**で確度を上げる（Prep 丸読みより m4:617/631 と問題演習）
- M1 S03 (**effort** ・モデル family) ・ M4 S10A/B (routing ・ tradeoff) は Exam1 で弱点 →Exam2 で克服済み。維持

◎ D2 Apps — 安定維持（配点33%・最大）

- 正答率85%で安定だが**題数が最多（34問）**。1問の配点的価値が最大。Worktrees（Q47）は P0 の公式 doc に対応。他は散発

◎ D6 Prompt — Structured Outputs の境界

- Q51 (output_config.format ・ casing ・ stop_reason ・ prefill 不可) は P0①の M2 m2:121-141 で潰す。context editing（Q18）も P0②で対応

● P2 — メンテナンス・スポット 必要時

- **D8 Tools/MCPs:** Exam1 50%→Exam2 100%。M3 S12-14 (MCP primitives ・ transports) は Exam1 で読んだ前提。忘却曲線で近いうちに軽く復習（詳細は [Exam1分析](#) 参照）
- **D7 Security:** Exam1 75%→Exam2 100%。M4 S14-16 は Exam1 で読んだ前提。維持
- **D3 Claude Code:** 題数2問のみ。print mode（Q20）は P0②で対応

▶▶ スキップ可

- **D4 Eval（100%）:** 題数1問・重み2.6%・両試験で正解。深掘り不要

受験技法 — Silent miss & 多選択対策

Silent miss 対策 (Exam2: 4問 / Exam1: 6問)

Silent miss = 「自信があったが誤」。本番で最も危険。**自信があるほど選択肢を全つぶし**する習慣が唯一の対策。

- ① 「自明」に見えたら警鐘 — Q6/20/44/47 はいずれも「これは分かる」と踏んで最もらしい選択肢を選んだ。最もらしいほど要件との照合を怠らない
- ② 全選択肢に「なぜこれはダメか」を一言ずつ — 説明不能な選択肢が残ればそれが正解候補
- ③ "best fits" / "Which TWO" で止まらない — 1つ選んで安心せず、残りを全評価

多選択「2問目漏れ」対策 (継続)

Exam1 で 5/10 問・Exam2 では軽減傾向 (Q20 のみ複選で1つ正解→もう1つ漏選)。本番でも継続警戒。

- ① 確実に正解を1つ選んだら止まらない
- ② 残り全選択肢に「これも正解になり得るか」を問う — 特に「～も要件を満たす」「～も同様に有効」の並列構造に警戒
- ③ "Which TWO" を再確認 — 1つしか選んでなければ必ずもう1つある

次のアクション

- 本日～: P0① (Prep M2 徹底・約60分) → P0② (公式 doc 3トピック・約45分)
- その後: **Exam3** を受験 (同パック最終)。D1/D6 の正答率が上がったか検証
- **Exam3** 後: 本 HTML フォーマットで 3 試験統合分析 (159 問)
- **本番 (720/1000)** — 現在 2 試験平均 82% で通過線 +10pt。D1 (69%) を 80% まで引き上げれば本番 720 は十分狙える
- 本番前日に本 HTML の「Part C トピックグループ」と「Part D P0」を総ざらい

現状地力: 配点の大きい D2(85%)+D5(89%)+D1(69%) = 重み 64.6% で加重平均 約 82%。D1 を 69%→80% に上げると加重平均 +1.6pt。最大の伸び代は **D1 (知識が浅い=学習で確実に上がる)**。D5/D2 は確信不足なので演習量、D1 は Prep 精読で質を変える。

データソース: `Exam2/{Results,incorrect,marked,all}.webarchive` (python3.12 plistlib 抽出) ・ `tasks/prepare-course-raw/m{2,3,4}-*.md` (grep で行番号確定)
姉妹: Exam1 弱点分析・Exam1 品質評価・タスク管理